AI VOICE BOT 4 U - MASTER PROJECT GUIDE (ONE-DAY ONBOARDING)
Version: 1.0
Updated: 2026-04-12
Owner: AI Voice Bot 4 U Team

0) WHY THIS FILE EXISTS

This is the single document to bring a new engineer/operator from zero context to productive deployment and support in one day.

If someone follows this file end-to-end, they should be able to:
1. Understand the product and architecture
2. Run locally
3. Deploy to AWS Lightsail
4. Validate live flow
5. Troubleshoot common issues
6. Operate safely with cost awareness

1) PRODUCT OVERVIEW

Product name: AI Voice Bot 4 U
Main purpose:
- Run live AI voice demos from browser microphone
- Place outbound calls through telecom providers
- Capture lead details, transcripts, and session summaries
- Support multiple client/project configurations

Primary user journeys:
A) Guest website visitor fills demo details and starts live demo
B) Authenticated user uses dashboard for calls, analytics, and config
C) Team deploys updates to Lightsail with one command

2) HIGH-LEVEL ARCHITECTURE

Core stack:
- Backend: FastAPI (Python)
- Real-time voice: Gemini Live API
- UI: Static HTML/CSS/JS
- Auth + lead DB: SQLite
- Client configs: File-based (optional MySQL/Mongo)
- Call state: In-memory (optional Redis for scale)

High-level flow (browser demo):
Browser -> /api/demo/options -> workspace/session assignment
Browser -> /ws/browser-audio/{session_key} (audio stream)
Browser -> /api/session/start (transport=browser)
Backend session engine -> Gemini Live -> response audio back to browser

High-level flow (telephony):
Dashboard -> /api/telephony/call
Backend creates pending call + provider callback routes
Provider webhook/media websocket -> backend media bridge
Session engine runs per call session key

3) CODE STRUCTURE (MOST IMPORTANT FILES)

```
================================================================
Backend entry:
- voice_sales_agent/web_app.py

Session and model logic:
- voice_sales_agent/session.py
- voice_sales_agent/gemini_api.py

Telephony providers:
- voice_sales_agent/telephony.py

State and analytics:
- voice_sales_agent/web_state.py
- voice_sales_agent/analytics.py
- voice_sales_agent/call_state.py

Auth and lead storage:
- voice_sales_agent/auth_backend.py

Configuration loader:
- voice_sales_agent/config.py

Frontend pages:
- voice_sales_agent/web_static/landing.html
- voice_sales_agent/web_static/index.html

Deployment assets:
- scripts/deploy_lightsail.sh
- deploy/lightsail/voice-sales-agent.service
- deploy/lightsail/nginx-voice-sales-agent.conf
- deploy/lightsail/rsync-excludes.txt

Project docs:
- docs/project_handbook.md
- docs/lightsail_deployment.md


================================================================
4) CURRENT SECURITY / ACCESS BEHAVIOR
================================================================
1. Signed cookie session auth for protected pages/APIs
2. Workspace isolation per authenticated user
3. Guest demo Start is gated:
   - Name required
   - Phone required
   - Email required
   - Backend enforces this; frontend-only bypass is not enough
4. Landing Start validation focus order:
   Name -> Phone -> Email

Known gap to fix:
- Provider signature hardening should be completed for all webhooks


================================================================
5) DESIGN AND UX DIRECTION
================================================================
Landing page goals:
- Explain value proposition quickly
- Collect visitor demo details
- Allow immediate Start/Stop voice interaction
- Show confidence/social proof/FAQ
```

Dashboard goals:
- Operational control center
- Start/Stop sessions
- Place calls
- See workspace metrics and recent calls
- Edit client/project behavior

Design strengths:
- Clear call-to-action flow
- Strong visual identity
- In-page demo removes friction

Design tradeoffs:
- Large static pages are harder to maintain
- JS logic is dense and can regress without tests

6) PROS AND CONS

Pros:
1. End-to-end functional system in production
2. Multi-provider telephony abstraction
3. Flexible client/project model
4. One-command deployment exists
5. Guest lead capture integrated with live demo

Cons:
1. web_app.py is very large (many concerns in one place)
2. Frontend files are monolithic
3. Limited automated tests for critical flows
4. Operational maturity is medium (manual checks still required)
5. Security hardening still has gaps in webhook validation

7) ONE-DAY ONBOARDING PLAN (TIMED)

Hour 1: Architecture and code map
- Read sections 1-4 of this guide
- Open key files listed in section 3

Hour 2: Local environment setup
- Create venv
- Install dependencies
- Configure .env
- Run dashboard locally

Hour 3: Core functional walkthrough
- Landing demo start/stop
- Dashboard login and workspace behavior
- Session artifacts inspection

Hour 4: Telephony understanding
- Read provider setup docs
- Follow one provider path (Twilio first is easiest)

Hour 5: Deployment pipeline
- Read deploy script and excludes
- Run dry-run deploy check

Hour 6: Live deployment + verification
- Deploy to Lightsail
- Verify service, domain, and key endpoints

Hour 7: Troubleshooting drills
- Simulate missing mic permission
- Simulate missing provider credentials
- Validate expected error messages

Hour 8: Handover readiness
- Update this guide with any findings
- Capture open risks + next actions

8) LOCAL SETUP (COMMANDS)

From project root:

1) Create virtual environment (if needed)
```
python3 -m venv .venv
```

2) Activate
```
source .venv/bin/activate
```

3) Install dependencies
```
pip install -r requirements.txt
```

4) Configure env
```
cp .env.example .env
# Fill at least GEMINI_API_KEY and basic settings
```

5) Run dashboard
```
python scripts/run_dashboard.py
```

6) Open
```
http://127.0.0.1:8000
```

9) DEPLOYMENT (PRODUCTION)

Current production target:
- Domain: https://aivoicebot4u.com
- Lightsail instance: voice-agent-prod
- App dir: /opt/new_voice_agent
- systemd service: voice-sales-agent.service

Deployment command:
```
./scripts/deploy_lightsail.sh
```

What it does:
1. Local syntax checks
2. Fetch temporary Lightsail SSH access details
3. rsync to /tmp/new_voice_agent_sync on server
4. rsync apply to /opt/new_voice_agent (with excludes)
5. Install requirements
6. Restart service
7. Print service status

After deploy verify:
1. systemd active

2. Nginx config valid
3. Domain responds
4. Landing changes visible
5. Start/Stop flow works

10) OPERATIONS RUNBOOK

Useful commands:

App logs:
```
sudo journalctl -u voice-sales-agent.service -f
```

Service status:
```
sudo systemctl status voice-sales-agent.service
```

Restart app:
```
sudo systemctl restart voice-sales-agent.service
```

Nginx check:
```
sudo nginx -t
```

Nginx logs:
```
sudo tail -f /var/log/nginx/error.log /var/log/nginx/access.log
```

11) TROUBLESHOOTING PLAYBOOK

Issue: Start button does nothing
Check:
1. Browser console errors
2. Mic permission prompt accepted?
3. WS connect to /ws/browser-audio/{session_key}
4. /api/session/start response body

Issue: "Browser audio channel is not connected"
- WebSocket failed or closed before Start API call
- Reopen page, ensure HTTPS, retry Start

Issue: "Authentication required"
- Session cookie missing/expired
- Re-login or recreate guest demo options flow

Issue: Call not placed
- Verify TELEPHONY_PROVIDER and provider creds in .env
- Check provider-specific response error code
- Twilio trial often blocks unverified destination numbers

Issue: Gemini model not responding
- Verify GEMINI_API_KEY
- Verify model names in .env
- Check app logs for Gemini SDK/model validation errors

Issue: Domain returns unexpected site
- DNS not pointing to Lightsail static IP
- Confirm authoritative nameservers and A record

12) COST CONTROL

```
================================================================
Lightsail instance plan (example used): small_3_0 (~$12/month).

Important:
- Stopped instance can still incur plan cost.
- Near-zero requires deleting chargeable resources.

To minimize cost now:
1. Stop instance (short-term pause)
2. Delete instance for true stop of plan billing
3. Release static IP if not needed
4. Remove unused snapshots/disks/load balancers


================================================================
13) RISK REGISTER (CURRENT)
================================================================
1. Monolithic backend/frontend files increase regression risk
2. Provider API changes can break call paths unexpectedly
3. Lack of strong automated tests for critical runtime flows
4. In-memory state default can fail under multi-worker scale
5. Data retention/compliance policy not yet formalized


================================================================
14) RECOMMENDED IMPROVEMENTS
================================================================
P0 (must-do):
1. Full webhook signature validation + replay protection
2. Smoke test suite for guest start gate + browser audio + call create
3. Rollback script and deploy health gates

P1 (high value):
1. Split web_app.py into modular routers/services
2. Split large frontend JS into modules
3. Add CI: lint + compile + smoke checks

P2 (scale/product):
1. Redis default for shared call state in multi-worker deployments
2. Observability dashboard for failure rates and latency
3. Backup/recovery policy for auth + lead data
4. Secrets management improvement (beyond .env)


================================================================
15) PRE-DEPLOY CHECKLIST
================================================================
[ ] .env validated for target environment
[ ] Required provider credentials present
[ ] Python compile checks pass
[ ] Deploy dry-run reviewed
[ ] No accidental local runtime files included
[ ] Service and nginx health confirmed


================================================================
16) POST-DEPLOY CHECKLIST
================================================================
[ ] Service active after restart
[ ] Domain reachable over HTTPS
[ ] Landing Start gate works (Name->Phone->Email)
[ ] Demo can start/stop with mic permission
```

[ ] Call API path tested (if provider enabled)
[ ] No critical errors in logs

17) CHANGELOG RULE (DO NOT SKIP)

For every major change, update this guide with:
1. What changed
2. Why it changed
3. Risk introduced
4. Rollback plan
5. Follow-up actions

If the team keeps this discipline, project memory loss is minimized.

END OF MASTER GUIDE